

Algorithmic Trading Infrastructure Build Plan

Prediction Markets + Options Execution System

1. Product Goal

Build an always-running algorithmic trading system that can:

1. Collect market data from prediction markets and options broker APIs.
2. Estimate probabilities or theoretical values.
3. Decide whether a trade has positive expected value.
4. Check risk limits before every order.
5. Execute trades through brokerage/exchange APIs.
6. Track orders, fills, positions, PnL, and errors.
7. Provide a simple dashboard and kill switch.
8. Run cheaply on a VPS using Docker Compose.

This is not a high-frequency trading system. The goal is reliability, observability, and safe automation.

The system should initially trade only personal capital. Do not manage outside money, client capital, or paid recommendations without legal/compliance review.

2. Core Architecture

The trading system should be separated into layers:

```
Market Data
↓
Prediction / Pricing Model
↓
Decision Engine
↓
Risk Engine
↓
Execution Engine
↓
Broker / Exchange API
↓
Portfolio + Logs + Monitoring
```

The most important rule:

The strategy/model should never directly place trades.

Bad architecture:

```
if signal == "buy":  
    broker.place_order(...)
```

Good architecture:

```
Strategy creates signal  
Decision engine creates trade intent  
Risk engine approves/rejects  
Execution engine places order  
Reconciliation engine verifies broker state
```

This prevents one bad model output from directly placing unsafe trades.

3. Recommended Tech Stack

Version 1 Stack

```
Python:  
  research  
  backtesting  
  ML/probability models  
  strategy experiments
```

```
Go:  
  production services  
  market data workers  
  risk engine  
  execution engine  
  dashboard API  
  alerts  
  reconciliation
```

```
Postgres:  
  market snapshots  
  signals  
  trade intents  
  orders
```

```
fills
positions
logs
risk events
```

Docker Compose:
run all services on one VPS

VPS:
DigitalOcean Droplet, AWS Lightsail, Hetzner, Linode, or Vultr

Why This Stack

```
Python thinks.
Go operates.
C++ accelerates later.
```

Python is best for research, modeling, and fast iteration.

Go is best for reliable always-running services, APIs, concurrency, logs, workers, and deployment.

C++ should only be added later for performance-heavy modules like options pricing, Monte Carlo simulation, high-speed backtesting, or order book simulation.

4. Deployment Target

Use a cheap VPS first.

Recommended:

```
DigitalOcean Droplet
Ubuntu
2 GB RAM preferred
Docker Compose
Postgres on same VPS
Go backend services
Python model service/scripts
Telegram or Discord alerts
```

Estimated cost:

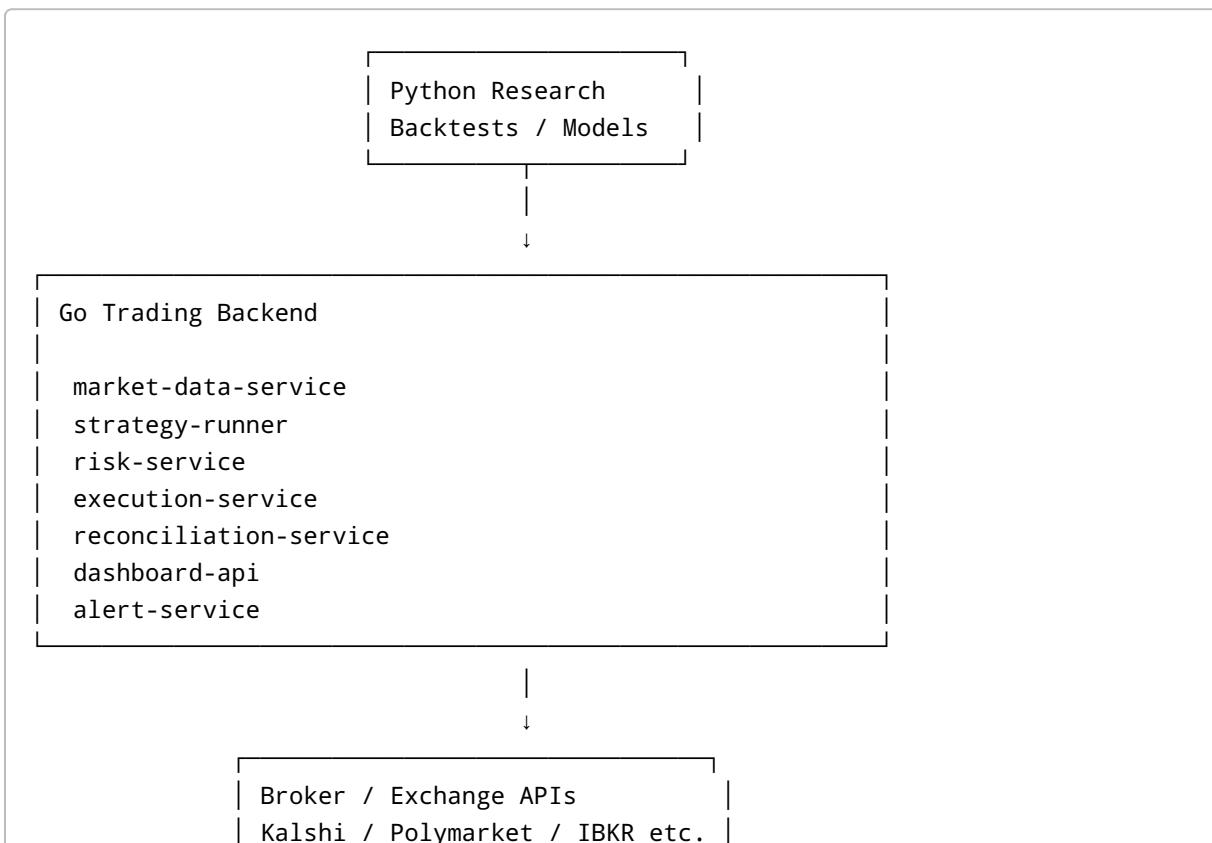
VPS: about \$6-\$12/month
Database: free if Postgres runs on the VPS
Dashboard: free
Alerts: free
Domain: optional

Avoid using these for live trading workers:

Vercel
Netlify
Google Colab
Jupyter notebooks
GitHub Actions scheduled jobs
free hosting that sleeps
your laptop

Those are fine for demos or research, but not for a live system that can place trades.

5. High-Level System Diagram





6. Main Services

6.1 Market Data Service

Purpose:

Collect market data from prediction markets and options APIs.

Responsibilities:

```
fetch available markets
fetch latest prices
fetch order books
fetch option chains
fetch quotes
store snapshots
handle API failures
respect rate limits
log raw responses when useful
```

Example output stored in database:

```
{
  "venue": "kalshi",
  "market_id": "EVENT_MARKET_YES",
  "yes_bid": 0.61,
  "yes_ask": 0.63,
  "no_bid": 0.37,
  "no_ask": 0.39,
  "volume": 12450,
  "open_interest": 8800,
  "timestamp": "2026-06-12T15:00:00Z"
}
```

6.2 Strategy Runner

Purpose:

Turn market data into signals.

For prediction markets:

```
market price = implied probability
model probability = your estimated true probability
edge = model probability - market price
```

Example:

```
YES ask price = 0.62
Model probability = 0.72
Edge = 0.10
```

The strategy runner does not place trades. It only emits signals or trade intents.

Example signal:

```
{
  "venue": "kalshi",
  "market_id": "EVENT_MARKET_YES",
  "side": "YES",
  "action": "BUY",
  "model_probability": 0.72,
  "market_price": 0.62,
  "edge": 0.10,
  "confidence": 0.80,
  "reason": "model probability exceeds market price after estimated costs"
}
```

For options:

```
theoretical option value = your model's estimated fair value
market ask = price required to buy
edge = theoretical value - market ask
```

Example:

```
Option ask = $2.10
Theoretical value = $2.45
Edge = $0.35
```

6.3 Decision Engine

Purpose:

Convert signals into trade intents.

The decision engine asks:

```
Is the edge large enough?
Is the market liquid enough?
Is the spread acceptable?
Is the model confidence high enough?
Is the event/asset allowed?
Is this market too close to settlement/expiration?
```

Prediction market expected value:

```
EV(YES) = p_model × (1 - price) - (1 - p_model) × price - costs
```

Where:

```
p_model = estimated probability of event happening
price = YES contract price
costs = fees + spread + slippage estimate
```

For options:

```
EV-like edge = theoretical_value - executable_price - costs
```

The decision engine should output:

```
{
  "trade_intent_id": "uuid",
  "venue": "kalshi",
  "market_id": "EVENT_MARKET_YES",
```

```
"action": "BUY",  
"side": "YES",  
"limit_price": 0.62,  
"quantity": 10,  
"max_loss": 6.20,  
"reason": "positive edge after costs"  
}
```

6.4 Risk Engine

Purpose:

Approve, reject, or resize every trade before execution.

The risk engine is the most important service.

It should check:

```
Is trading enabled?  
Is the kill switch off?  
Is this venue enabled?  
Is this strategy enabled?  
Is the trade size allowed?  
Is the daily loss limit hit?  
Is total exposure too high?  
Is this a duplicate order?  
Is there already an open position?  
Is the price sane?  
Is the market too illiquid?  
Is the spread too wide?  
Is there enough cash?  
Is the order type allowed?
```

Example risk limits:

```
MAX_DAILY_LOSS = $50  
MAX_POSITION_SIZE = $25  
MAX_TOTAL_EXPOSURE = $250  
MAX_OPEN_ORDERS = 5  
MAX_SPREAD_PCT = 10%  
MIN_EDGE = 5%  
TRADING_ENABLED = false by default
```


Risk engine response:

```
{
  "status": "APPROVED",
  "approved_quantity": 10,
  "approved_limit_price": 0.62,
  "risk_notes": []
}
```

Or:

```
{
  "status": "REJECTED",
  "reason": "daily loss limit reached"
}
```

The execution engine should only accept approved orders.

6.5 Execution Engine

Purpose:

Send approved orders to broker/exchange APIs.

Responsibilities:

```
place limit orders
cancel orders
modify orders if supported
track order status
handle partial fills
handle rejected orders
handle API errors
prevent duplicate submissions
log raw API responses
```

Important execution rules:

```
Use limit orders, not market orders.
Never retry blindly.
Use idempotency keys when available.
```

Log every request and response.
Do not place orders if broker state cannot be verified.

Execution flow:

```
Approved trade intent
↓
Create local order record as PENDING_SUBMIT
↓
Send order to API
↓
Receive broker order ID
↓
Update local order as SUBMITTED
↓
Poll or stream order status
↓
Update as FILLED, PARTIAL, CANCELED, or REJECTED
```

6.6 Reconciliation Service

Purpose:

Ensure internal database state matches broker/exchange state.

This service should run periodically.

It checks:

```
broker cash vs internal cash
broker positions vs internal positions
broker open orders vs internal open orders
broker fills vs internal fills
```

If there is a mismatch:

```
pause live trading
send alert
write reconciliation error
require manual review
```

This prevents the bot from trading based on stale or incorrect internal state.

6.7 Dashboard API

Purpose:

Let you easily inspect and control the product.

Minimum dashboard pages:

1. Home / Status
 - bot running?
 - trading enabled?
 - last market data update
 - last signal
 - last order
 - current mode: paper or live
2. Portfolio
 - cash
 - open positions
 - realized PnL
 - unrealized PnL
 - total exposure
3. Orders
 - submitted orders
 - open orders
 - filled orders
 - canceled orders
 - rejected orders
4. Signals
 - market
 - model probability
 - market price
 - edge
 - decision
 - reason
5. Risk
 - daily loss
 - max loss limit
 - max exposure
 - rejected trades

- kill switch

6. Logs

- API errors
- failed orders
- reconciliation errors
- strategy errors

Most important dashboard control:

```
TRADING_ENABLED = false
```

The kill switch should be obvious and easy to use.

6.8 Alert Service

Purpose:

Notify you when something important happens.

Use Telegram, Discord, email, or SMS.

Send alerts for:

```
order submitted
order filled
order rejected
daily loss warning
kill switch activated
API error
broker reconciliation mismatch
bot crashed/restarted
large position opened
```

Example alert:

```
TRADE FILLED
```

```
Venue: Kalshi
Market: EVENT_MARKET_YES
Side: YES
Quantity: 10
```

Price: 0.62
Reason: edge=0.10, model_prob=0.72
Mode: PAPER

7. Folder Structure

Use one monorepo:

```
trading-system/  
  README.md  
  docker-compose.yml  
  .env.example  
  
python/  
  pyproject.toml  
  research/  
    notebooks/  
  models/  
    probability_model.py  
    options_model.py  
  backtests/  
    prediction_market_backtest.py  
    options_backtest.py  
  scripts/  
    generate_signals.py  
  
go/  
  go.mod  
  go.sum  
  
cmd/  
  market-data-service/  
    main.go  
  strategy-runner/  
    main.go  
  risk-service/  
    main.go  
  execution-service/  
    main.go  
  reconciliation-service/  
    main.go  
  dashboard-api/  
    main.go  
  alert-service/
```

```
main.go

internal/
  config/
    config.go

  database/
    db.go

  venues/
    kalshi/
      client.go
      market_data.go
      orders.go
    polymarket/
      client.go
      market_data.go
      orders.go
    ibkr/
      client.go
      market_data.go
      orders.go

  strategy/
    signal.go
    decision.go

  risk/
    limits.go
    checks.go
    sizing.go
    kill_switch.go

  execution/
    order.go
    order_state_machine.go
    executor.go

  portfolio/
    positions.go
    pnl.go

  reconciliation/
    reconcile.go

  alerts/
    telegram.go
    discord.go
```

```
    api/  
      routes.go  
      handlers.go  
  
    db/  
      migrations/  
        001_init.sql  
        002_orders.sql  
        003_positions.sql  
  
    docs/  
      architecture.md  
      deployment.md  
      risk.md
```

8. Database Schema

8.1 Venues

```
CREATE TABLE venues (  
  id SERIAL PRIMARY KEY,  
  name TEXT UNIQUE NOT NULL,  
  enabled BOOLEAN NOT NULL DEFAULT false,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

8.2 Markets

```
CREATE TABLE markets (  
  id SERIAL PRIMARY KEY,  
  venue TEXT NOT NULL,  
  market_id TEXT NOT NULL,  
  symbol TEXT,  
  title TEXT,  
  status TEXT,  
  close_time TIMESTAMP,  
  raw JSONB,  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),
```

```
    UNIQUE (venue, market_id)
);
```

8.3 Market Snapshots

```
CREATE TABLE market_snapshots (
    id BIGSERIAL PRIMARY KEY,
    venue TEXT NOT NULL,
    market_id TEXT NOT NULL,
    bid DOUBLE PRECISION,
    ask DOUBLE PRECISION,
    mid DOUBLE PRECISION,
    last DOUBLE PRECISION,
    volume DOUBLE PRECISION,
    open_interest DOUBLE PRECISION,
    raw JSONB,
    captured_at TIMESTAMP DEFAULT NOW()
);
```

8.4 Signals

```
CREATE TABLE signals (
    id BIGSERIAL PRIMARY KEY,
    venue TEXT NOT NULL,
    market_id TEXT NOT NULL,
    strategy_name TEXT NOT NULL,
    action TEXT NOT NULL,
    side TEXT,
    model_probability DOUBLE PRECISION,
    market_price DOUBLE PRECISION,
    edge DOUBLE PRECISION,
    confidence DOUBLE PRECISION,
    reason TEXT,
    raw JSONB,
    processed BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);
```


8.5 Trade Intents

```
CREATE TABLE trade_intents (  
  id UUID PRIMARY KEY,  
  signal_id BIGINT REFERENCES signals(id),  
  venue TEXT NOT NULL,  
  market_id TEXT NOT NULL,  
  action TEXT NOT NULL,  
  side TEXT,  
  limit_price DOUBLE PRECISION NOT NULL,  
  quantity DOUBLE PRECISION NOT NULL,  
  max_loss DOUBLE PRECISION,  
  status TEXT NOT NULL DEFAULT 'PENDING_RISK',  
  reason TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

8.6 Risk Decisions

```
CREATE TABLE risk_decisions (  
  id BIGSERIAL PRIMARY KEY,  
  trade_intent_id UUID REFERENCES trade_intents(id),  
  decision TEXT NOT NULL,  
  approved_quantity DOUBLE PRECISION,  
  approved_limit_price DOUBLE PRECISION,  
  reason TEXT,  
  checks JSONB,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

8.7 Orders

```
CREATE TABLE orders (  
  id UUID PRIMARY KEY,  
  trade_intent_id UUID REFERENCES trade_intents(id),  
  venue TEXT NOT NULL,  
  market_id TEXT NOT NULL,  
  broker_order_id TEXT,  
  action TEXT NOT NULL,  
  side TEXT,
```

```
order_type TEXT NOT NULL,  
limit_price DOUBLE PRECISION,  
quantity DOUBLE PRECISION NOT NULL,  
filled_quantity DOUBLE PRECISION DEFAULT 0,  
avg_fill_price DOUBLE PRECISION,  
status TEXT NOT NULL,  
raw_request JSONB,  
raw_response JSONB,  
created_at TIMESTAMP DEFAULT NOW(),  
updated_at TIMESTAMP DEFAULT NOW()  
);
```

Order statuses:

```
PENDING_SUBMIT  
SUBMITTED  
PARTIALLY_FILLED  
FILLED  
CANCELED  
REJECTED  
FAILED
```

8.8 Fills

```
CREATE TABLE fills (  
  id BIGSERIAL PRIMARY KEY,  
  order_id UUID REFERENCES orders(id),  
  venue TEXT NOT NULL,  
  broker_fill_id TEXT,  
  price DOUBLE PRECISION NOT NULL,  
  quantity DOUBLE PRECISION NOT NULL,  
  fee DOUBLE PRECISION DEFAULT 0,  
  raw JSONB,  
  filled_at TIMESTAMP NOT NULL  
);
```

8.9 Positions

```
CREATE TABLE positions (  
  id BIGSERIAL PRIMARY KEY,
```

```
venue TEXT NOT NULL,  
market_id TEXT NOT NULL,  
side TEXT,  
quantity DOUBLE PRECISION NOT NULL,  
avg_price DOUBLE PRECISION NOT NULL,  
realized_pnl DOUBLE PRECISION DEFAULT 0,  
unrealized_pnl DOUBLE PRECISION DEFAULT 0,  
updated_at TIMESTAMP DEFAULT NOW(),  
UNIQUE (venue, market_id, side)  
);
```

8.10 System State

```
CREATE TABLE system_state (  
    key TEXT PRIMARY KEY,  
    value TEXT NOT NULL,  
    updated_at TIMESTAMP DEFAULT NOW()  
);
```

Initial values:

```
INSERT INTO system_state (key, value)  
VALUES  
( 'TRADING_ENABLED', 'false' ),  
( 'MODE', 'paper' ),  
( 'KILL_SWITCH', 'false' );
```

9. Go Service Design

9.1 Common Config

Every Go service should load:

```
DATABASE_URL  
MODE  
TRADING_ENABLED  
KALSHI_API_KEY  
KALSHI_API_SECRET  
POLYMARKET_API_KEY
```

```
IBKR_HOST
IBKR_PORT
TELEGRAM_BOT_TOKEN
TELEGRAM_CHAT_ID
```

Never commit `.env`.

Use `.env.example` for placeholders.

9.2 Market Data Service Pseudocode

```
func main() {
    cfg := LoadConfig()
    db := ConnectDB(cfg.DatabaseURL)
    kalshi := NewKalshiClient(cfg)

    ticker := time.NewTicker(10 * time.Second)

    for range ticker.C {
        markets, err := kalshi.GetMarkets()
        if err != nil {
            LogError(err)
            continue
        }

        for _, market := range markets {
            snapshot, err := kalshi.GetMarketSnapshot(market.ID)
            if err != nil {
                LogError(err)
                continue
            }

            SaveMarketSnapshot(db, snapshot)
        }
    }
}
```

9.3 Strategy Runner Pseudocode

```
func main() {
    db := ConnectDB()
    ticker := time.NewTicker(30 * time.Second)
```

```

for range ticker.C {
    snapshots := LoadLatestSnapshots(db)

    for _, snapshot := range snapshots {
        signal := EvaluatePredictionMarket(snapshot)

        if signal.Edge > MinEdge {
            SaveSignal(db, signal)
        }
    }
}

```

For V1, strategy logic can be simple and written in Go.

Later, Go can call Python:

```

Go loads market snapshot
Go sends snapshot to Python model
Python returns probability
Go stores signal

```

9.4 Risk Service Pseudocode

```

func ApproveTrade(intent TradeIntent, portfolio Portfolio, limits RiskLimits)
RiskDecision {
    if !limits.TradingEnabled {
        return Reject("trading disabled")
    }

    if limits.KillSwitch {
        return Reject("kill switch enabled")
    }

    if portfolio.DailyPnL <= -limits.MaxDailyLoss {
        return Reject("daily loss limit reached")
    }

    if intent.MaxLoss > limits.MaxLossPerTrade {
        return Reject("max loss per trade exceeded")
    }
}

```

```

    if portfolio.TotalExposure + intent.MaxLoss > limits.MaxTotalExposure {
        return Reject("max total exposure exceeded")
    }

    if IsDuplicateOrder(intent) {
        return Reject("duplicate order")
    }

    return Approve(intent.Quantity, intent.LimitPrice)
}

```

9.5 Execution Service Pseudocode

```

func main() {
    db := ConnectDB()
    executor := NewExecutor()

    for {
        approvedOrders := LoadApprovedTradeIntents(db)

        for _, intent := range approvedOrders {
            localOrder := CreatePendingOrder(db, intent)

            response, err := executor.PlaceLimitOrder(intent)
            if err != nil {
                MarkOrderFailed(db, localOrder.ID, err)
                SendAlert("Order failed")
                continue
            }

            MarkOrderSubmitted(db, localOrder.ID, response.BrokerOrderID,
response.Raw)
            SendAlert("Order submitted")
        }

        time.Sleep(2 * time.Second)
    }
}

```

9.6 Reconciliation Service Pseudocode

```
func main() {  
    db := ConnectDB()  
    broker := NewBrokerClient()  
  
    ticker := time.NewTicker(1 * time.Minute)  
  
    for range ticker.C {  
        brokerPositions := broker.GetPositions()  
        localPositions := LoadLocalPositions(db)  
  
        mismatch := ComparePositions(brokerPositions, localPositions)  
  
        if mismatch {  
            SetTradingEnabled(db, false)  
            SendAlert("Reconciliation mismatch. Trading disabled.")  
            SaveReconciliationError(db)  
        }  
    }  
}
```

10. Python Model Layer

Python should be used for:

```
research  
backtesting  
modeling  
probability estimation  
options pricing experiments  
ML
```

Example prediction model interface:

```
def estimate_probability(market_data: dict) -> dict:  
    """  
    Returns model probability and confidence.  
    """  
    probability = 0.72  
    confidence = 0.80
```

```
return {  
    "model_probability": probability,  
    "confidence": confidence,  
    "reason": "example model output"  
}
```

For V1, you can use simple models:

```
manual probabilities  
simple rules  
market-implied changes  
poll/news-based model  
historical base rates  
logistic regression  
calibrated classifier
```

Do not start with a complex LLM trading agent. Start with a transparent probability model.

11. Prediction Market Strategy Logic

Core Idea

Prediction market prices are probabilities.

```
YES price = implied probability of YES  
NO price = implied probability of NO
```

Example:

```
YES ask = 0.62  
Market implies about 62%  
Your model says 72%  
Edge = 10%
```

Trade Rule

```
If model_probability - yes_ask > required_edge:  
    buy YES
```



```
If no_ask - model_probability > required_edge:  
    buy NO
```

More explicit:

```
def prediction_market_decision(yes_ask, no_ask, model_probability, costs):  
    yes_edge = model_probability - yes_ask  
    no_edge = (1 - model_probability) - no_ask  
  
    min_edge = 0.05 + costs  
  
    if yes_edge > min_edge:  
        return "BUY_YES"  
  
    if no_edge > min_edge:  
        return "BUY_NO"  
  
    return "SKIP"
```

Position Sizing

Start conservative.

```
Max position per market: 1%-2% of portfolio  
Max total exposure: 10%-25% of portfolio  
Max daily loss: 1%-2% of portfolio
```

Simple sizing:

```
def size_position(portfolio_value, edge, confidence):  
    base_size = portfolio_value * 0.01  
  
    if edge > 0.10 and confidence > 0.8:  
        return base_size * 2  
  
    return base_size
```

Do not use full Kelly sizing in V1. It can produce aggressive position sizes.

12. Options Strategy Logic

Options are harder than prediction markets because you are trading a payoff curve, volatility, time decay, and liquidity.

Start with scanning and alerts before live automated options execution.

Options Data Needed

```
underlying price
option chain
bid
ask
mid
implied volatility
delta
gamma
theta
vega
expiration
strike
open interest
volume
spread
```

Basic Filters

```
avoid wide spreads
avoid low open interest
avoid low volume
avoid near-expiration contracts at first
avoid naked short options
avoid complex multi-leg execution until later
```

Example filters:

```
min open interest: 500
max bid/ask spread: 10%
min days to expiration: 7
max position size: 1 contract
defined risk only
```

Options Decision Example

```
def options_decision(option, theoretical_value):
    spread = option.ask - option.bid
    spread_pct = spread / option.mid

    if spread_pct > 0.10:
        return "SKIP"

    if option.open_interest < 500:
        return "SKIP"

    edge = theoretical_value - option.ask

    if edge > 0:
        return "BUY_TO_OPEN"

    return "SKIP"
```

For V1, options should be:

```
scanner first
paper trading second
manual approval third
small live automation last
```

13. Safety Rules

Trading Mode

The system should support:

```
MODE=paper
MODE=live
```

Default:

```
MODE=paper
TRADING_ENABLED=false
```

The bot should never live trade unless both are true:

```
MODE=live
TRADING_ENABLED=true
KILL_SWITCH=false
```

Kill Switch

The kill switch should:

```
disable all new orders
optionally cancel open orders
send alert
write event to database
require manual restart
```

Example:

```
UPDATE system_state SET value = 'true' WHERE key = 'KILL_SWITCH';
UPDATE system_state SET value = 'false' WHERE key = 'TRADING_ENABLED';
```

Required Risk Limits

Minimum risk limits:

```
Max loss per trade
Max daily loss
Max total exposure
Max orders per day
Max open orders
Max exposure per venue
Max exposure per market
Max position size
Max spread allowed
Minimum edge required
Allowed symbols/markets
Blocked symbols/markets
```

Failure Behavior

If something weird happens, the bot should stop trading.

Stop trading on:

```
database connection failure
broker API authentication failure
portfolio mismatch
order status unknown
duplicate order detected
daily loss reached
uncaught panic in execution service
```

The system should be conservative:

```
When uncertain, do not trade.
```

14. Docker Compose Setup

Example `docker-compose.yml`:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: trading
      POSTGRES_USER: trading_user
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  market-data-service:
    build:
      context: ./go
      dockerfile: Dockerfile
    command: ./market-data-service
    env_file:
      - .env
```

```
    depends_on:
      - postgres

strategy-runner:
  build:
    context: ./go
    dockerfile: Dockerfile
  command: ./strategy-runner
  env_file:
    - .env
  depends_on:
    - postgres

risk-service:
  build:
    context: ./go
    dockerfile: Dockerfile
  command: ./risk-service
  env_file:
    - .env
  depends_on:
    - postgres

execution-service:
  build:
    context: ./go
    dockerfile: Dockerfile
  command: ./execution-service
  env_file:
    - .env
  depends_on:
    - postgres

reconciliation-service:
  build:
    context: ./go
    dockerfile: Dockerfile
  command: ./reconciliation-service
  env_file:
    - .env
  depends_on:
    - postgres

dashboard-api:
  build:
    context: ./go
    dockerfile: Dockerfile
  command: ./dashboard-api
```

```
ports:
  - "8080:8080"
env_file:
  - .env
depends_on:
  - postgres

volumes:
  postgres_data:
```

15. Go Dockerfile

Example `go/Dockerfile`:

```
FROM golang:1.22 AS builder

WORKDIR /app

COPY go.mod go.sum ./
RUN go mod download

COPY . .

RUN go build -o market-data-service ./cmd/market-data-service
RUN go build -o strategy-runner ./cmd/strategy-runner
RUN go build -o risk-service ./cmd/risk-service
RUN go build -o execution-service ./cmd/execution-service
RUN go build -o reconciliation-service ./cmd/reconciliation-service
RUN go build -o dashboard-api ./cmd/dashboard-api

FROM debian:bookworm-slim

WORKDIR /app

COPY --from=builder /app/market-data-service .
COPY --from=builder /app/strategy-runner .
COPY --from=builder /app/risk-service .
COPY --from=builder /app/execution-service .
COPY --from=builder /app/reconciliation-service .
COPY --from=builder /app/dashboard-api .

CMD [ "./dashboard-api" ]
```

16. Local Development Commands

Create repo:

```
mkdir trading-system  
cd trading-system  
git init
```

Start database:

```
docker compose up -d postgres
```

Run migrations:

```
psql "$DATABASE_URL" -f db/migrations/001_init.sql
```

Run Go service locally:

```
cd go  
go run ./cmd/market-data-service
```

Run all services:

```
docker compose up -d
```

View logs:

```
docker compose logs -f market-data-service  
docker compose logs -f execution-service  
docker compose logs -f dashboard-api
```

Stop everything:

```
docker compose down
```


17. VPS Deployment Plan

Step 1: Create VPS

Use:

```
Ubuntu 22.04 or 24.04
2 GB RAM preferred
SSH key authentication
```

Step 2: Install Docker

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo systemctl enable docker
sudo systemctl start docker
```

Step 3: Clone Repo

```
git clone <your-repo-url>
cd trading-system
```

Step 4: Add `.env`

```
nano .env
```

Example:

```
POSTGRES_PASSWORD=change_me
DATABASE_URL=postgres://trading_user:change_me@postgres:5432/trading?
sslmode=disable

MODE=paper
TRADING_ENABLED=false
KILL_SWITCH=false

KALSHI_API_KEY=
KALSHI_API_SECRET=
```

```
POLYMARKET_API_KEY=
```

```
TELEGRAM_BOT_TOKEN=
```

```
TELEGRAM_CHAT_ID=
```

Step 5: Start Services

```
docker compose up -d
```

Step 6: Check Logs

```
docker compose logs -f
```

Step 7: Open Dashboard

```
http://your-server-ip:8080
```

For V1, protect dashboard access by only allowing your IP or using basic auth.

Do not leave a trading dashboard open to the public internet.

18. Security Checklist

Minimum security:

```
Use SSH keys, not passwords.  
Never commit .env.  
Restrict dashboard access.  
Use paper mode by default.  
Use small API permissions if possible.  
Store API keys only on VPS.  
Rotate keys if exposed.  
Back up database.  
Log every order.  
Disable trading on reconciliation mismatch.  
Use limit orders only.
```

Better later:

Use a reverse proxy with HTTPS.
Add login to dashboard.
Use cloud secrets manager.
Use read-only API keys where possible.
Separate paper and live accounts.
Create deployment user with limited permissions.

19. Testing Plan

Unit Tests

Test:

expected value calculation
edge calculation
position sizing
risk checks
duplicate order detection
order state transitions
PnL calculations

Integration Tests

Test:

market data fetch
database insert
signal generation
risk approval
paper order execution
dashboard status
alert sending

Paper Trading Tests

Before live trading:

run for at least 1-2 weeks in paper mode
verify no duplicate orders

```
verify orders match signals
verify broker state matches database
verify kill switch works
verify alerts work
verify daily loss limit works
```

Live Tiny-Capital Tests

Start with:

```
very small position size
one venue
one strategy
one market type
limit orders only
manual monitoring
```

20. Build Milestones

Milestone 1: Database + Dashboard Skeleton

Goal:

```
Postgres is running.
Dashboard shows system status.
Can toggle paper/live and trading enabled/disabled.
```

Build:

```
system_state table
dashboard-api
basic HTML or JSON endpoints
kill switch endpoint
```

Milestone 2: Market Data Worker

Goal:

Market data is being collected and stored.

Build:

Kalshi market data client
market_snapshots table writes
logs
dashboard page showing latest prices

Milestone 3: Simple Strategy Runner

Goal:

Bot creates signals but does not trade.

Build:

basic probability input
edge calculation
signals table
dashboard page showing signals

Milestone 4: Risk Engine

Goal:

Every trade intent is approved or rejected based on rules.

Build:

risk limits
risk_decisions table
max position size
max daily loss
kill switch
duplicate order detection

Milestone 5: Paper Execution Engine

Goal:

Bot simulates trades without real money.

Build:

paper order creation
paper fills
paper positions
paper PnL
alerts

Milestone 6: Reconciliation

Goal:

System can compare internal state to exchange/broker state.

Build:

fetch real broker positions
compare local positions
pause trading on mismatch
send alert

Milestone 7: Live Execution With Tiny Size

Goal:

Bot can place real limit orders with very small size.

Rules:

```
one venue only
one strategy only
one small market universe
max loss per trade very small
daily loss limit very small
manual monitoring
```

Milestone 8: Options Scanner

Goal:

```
Options system scans and alerts but does not automatically trade.
```

Build:

```
option chain ingestion
liquidity filters
spread filters
theoretical value calculation
candidate trade alerts
```

Only automate options after prediction market execution is stable.

21. First Strategy To Implement

Start with prediction markets because they are simpler.

Strategy: Probability Edge

Inputs:

```
market_id
yes_bid
yes_ask
no_bid
no_ask
model_probability
```

```
fees estimate
slippage estimate
```

Decision:

```
yes_edge = model_probability - yes_ask
no_edge = (1 - model_probability) - no_ask
```

Trade:

```
if yes_edge > min_edge:
    buy YES

if no_edge > min_edge:
    buy NO

else:
    skip
```

Risk:

```
max $10 per trade at first
max $50 total exposure
max $20 daily loss
paper mode first
```

22. First Week Build Plan

Day 1

```
Create repo.
Create Docker Compose.
Start Postgres.
Create database schema.
Create Go dashboard-api.
Add system_state table.
Show trading status on dashboard.
```


Day 2

```
Create Kalshi client.  
Fetch markets.  
Store market snapshots.  
Show latest snapshots on dashboard.
```

Day 3

```
Create simple strategy-runner.  
Manually set model_probability.  
Generate signals.  
Store signals.  
Show signals on dashboard.
```

Day 4

```
Create trade_intents.  
Create risk-service.  
Implement kill switch, max trade size, max daily loss.  
Store risk decisions.
```

Day 5

```
Create paper execution-service.  
Simulate fills.  
Create positions table.  
Track paper PnL.  
Send Telegram/Discord alerts.
```

Day 6

```
Create reconciliation-service.  
Compare paper positions to internal state.  
Add dashboard pages for orders, fills, positions, logs.
```

Day 7

```
Run full paper trading loop.  
Fix bugs.  
Test kill switch.  
Test duplicate order protection.  
Test service restarts.
```

23. What Not To Build Yet

Avoid:

```
HFT  
Kubernetes  
complex microservices  
custom event bus  
C++ live execution path  
LLM directly placing trades  
naked options selling  
client money  
mobile app  
fancy frontend
```

Build the boring system first.

The boring system is what keeps you from losing money due to engineering mistakes.

24. Final Target

The first complete version should do this:

1. Runs on a cheap VPS.
2. Collects prediction market data.
3. Generates probability-based signals.
4. Calculates edge.
5. Creates trade intents.
6. Checks risk.
7. Executes paper trades.
8. Tracks orders/fills/positions/PnL.

9. Shows everything in a dashboard.
10. Sends alerts.
11. Has a kill switch.
12. Can later switch to tiny live trades.

The core product is:

An automated trading operating system that turns model beliefs into controlled, logged, risk-checked trades.

Build it in this order:

database
dashboard
market data
signals
risk
paper execution
reconciliation
alerts
tiny live execution
options scanner
C++ pricing/backtest module later